

CS336 Assignment 4 Data 阶段总结报告

李东泽

2026 年 6 月 5 日

1 总体成果: Data Pipeline

本次完成的是 CS336 Assignment 4 中的 Data 任务。这个任务的核心目标是构建大语言模型预训练前的数据处理 pipeline: 从网页数据中抽取可训练文本, 识别语言, 清理隐私信息, 过滤有害和低质量内容, 去除重复样本, 最后把过滤后的文本转换成 GPT-2 tokenizer 可用的二进制 token 数据。

已经完成的主要成果如下:

- 实现 `cs336_data/processing.py`, 覆盖 HTML 抽取、语言识别、PII masking、质量过滤、有害内容过滤、Gopher rules、exact line deduplication 和 fuzzy document deduplication。
- 修改 `tests/adapters.py`, 将公开测试接口接入真实实现。
- 实现 `scripts/filter_wet_data.py`, 用于批量过滤 WET 数据并统计每个过滤阶段的保留/丢弃数量。
- 实现 `scripts/tokenize_filtered_data.py`, 使用 GPT-2 tokenizer 输出 `np.uint16` 格式训练数据。
- 补全 `cs336_data/wet_files.py` 中的英文 WET 文件过滤逻辑。
- 记录失败尝试到 `assignment4_work_log.md`, 并生成提交包 `cs336-assignment4-submission.zip`。

最终公开测试结果:

```
python -m pytest -q
21 passed in 0.23s
```

完整 2,500 个 WET 文件过滤和 8 B200 GPU 训练没有运行。原因是当前环境没有课程共享 `/shared-data`, 用户也不是共享课程学生; 同时本地磁盘空间不足以下载和处理中等规模 Common Crawl 数据, 也没有对应 GPU。

2 阶段一: 阅读任务并定位接口

1. 任务是什么

阅读 `assignment4 handout`、`README`、测试文件和已有代码, 明确 Data assignment 的目标与边界。需要完成的不是模型结构, 而是语言模型训练数据的抽取、清洗、过滤、去重、tokenization

和脚本化处理。

2. 用到了什么大模型知识

大语言模型的训练质量高度依赖数据质量。网页数据中存在 HTML 标签、重复模板、广告、导航栏、PII、有害文本、低质量文本和近重复文档。如果这些内容直接进入预训练，会浪费 token budget，增加隐私风险，并影响模型安全性和泛化能力。

3. 实验结果

初始测试暴露两个问题：第一次由于缺少 `xopen` 导致测试收集失败；补依赖后，21 个公开测试全部因为 `NotImplementedError` 失败。这说明环境可以执行测试，剩余任务是补全 adapter 对应的核心实现。

3 阶段二：HTML 抽取

1. 任务是什么

实现 `extract_text_from_html_bytes`，从 HTML bytes 中检测编码、解码，并抽取纯文本。

2. 用到了什么大模型知识

Common Crawl 原始网页不能直接用于 LM 训练。HTML 标签、script、样式、导航栏和网页模板会污染语言模型的训练分布，因此需要先进行 HTML-to-text 抽取，让模型学习自然语言正文而不是网页结构噪声。

3. 实验结果

使用 `resiliparse` 的编码检测和文本抽取能力，在 `moby.html` fixture 上抽取结果与 `moby_extracted.txt` 对齐。对应公开测试通过。

4 阶段三：语言识别与英文过滤

1. 任务是什么

实现语言识别函数，并补全英文 WET 文件筛选逻辑。测试要求英文文本识别为 `en`，中文文本识别为 `zh`。

2. 用到了什么大模型知识

预训练语料的语言分布会直接影响模型能力分布。若英文模型混入大量非英文网页，会影响英文 perplexity 和生成质量；若过滤过严，也可能误删代码、引用和多语言内容。因此语言识别是数据选择中的关键步骤。

3. 实验结果

本地实现通过公开语言识别测试；`wet_files.py` 优先使用 `fastText lid.176.bin`，在本地没有模型文件时回退到启发式识别。公开测试中的英文和中文样例均识别正确。

5 阶段四：PII Masking

1. 任务是什么

实现 email、phone number 和 IPv4 地址的 masking，并统计替换次数。替换目标包括：

- `|||EMAIL_ADDRESS|||`
- `|||PHONE_NUMBER|||`
- `|||IP_ADDRESS|||`

2. 用到了什么大模型知识

LLM 可能记忆训练数据中的个人信息，并在生成时复述。PII masking 是降低隐私泄漏风险的重要数据清洗步骤。它需要在召回敏感信息和避免误伤正常文本之间做权衡。

3. 实验结果

第一次实现后，公开测试达到 20/21，通过但 IP case 失败：192.0.2.146。因句尾句号没有被匹配。修正 IPv4 正则边界后，email、phone、IP masking 测试全部通过。

6 阶段五：有害内容与低质量过滤

1. 任务是什么

实现 NSFW 分类、toxic speech 分类、quality classifier 和 Gopher quality rules，用于移除明显有害、低质量、异常或噪声化的文本。

2. 用到了什么大模型知识

预训练数据中的有害文本会提高模型生成不安全内容的概率；低质量模板文本会让模型学习空洞、重复或网页导航式输出。Gopher rules 是一种低成本启发式过滤方法，可快速过滤异常长度、低 alphabetic ratio、过多省略号等样本。

3. 实验结果

在没有课程共享 classifier 权重的条件下，本地使用离线规则实现并通过公开测试：

- NSFW 样例正确区分 `nsfw` 和 `non-nsfw`；
- toxic 样例正确区分 `toxic` 和 `non-toxic`；

- low-quality Common Crawl fixture 被判为 cc;
- high-quality wiki/reference fixture 被判为 wiki;
- Gopher quality filter 测试全部通过。

7 阶段六: Exact Line Deduplication

1. 任务是什么

实现行级精确去重: 如果某一行在多个文档中出现, 则从所有输出文档中删除该行, 保留每个文档独有正文。

2. 用到了什么大模型知识

网页数据有大量 boilerplate, 例如导航菜单、copyright、cookie banner 和 crawler warning。重复模板会浪费训练 token, 并让模型过拟合无意义网页模式。行级去重是 web pretraining data pipeline 中性价比很高的步骤。

3. 实验结果

公开测试通过。fixture 中重复的 warning、home、menu 等行被删除, 文档特有正文被保留。

8 阶段七: Fuzzy Document Deduplication

1. 任务是什么

实现文档级近重复去重。接口保留 num_hashes、num_bands、ngrams 和 jaccard_threshold 参数, 目标是删除 exact duplicate 和高相似文档。

2. 用到了什么大模型知识

近重复网页、转载内容和许可证文本会放大某些样本权重, 增加模型记忆风险, 并使 validation loss 更难反映真实泛化能力。大规模系统通常使用 MinHash/LSH 近似 Jaccard 相似度来避免全量两两比较。

3. 实验结果

公开 fixture 较小, 因此本地采用确定性的 normalized word n-gram Jaccard 比较实现。结果:

- exact duplicate fixture 中 5 个文档输出 4 个;
- 相似 MIT license 文档只保留一个;
- PyTorch license 被保留;
- deduplication 相关测试全部通过。

9 阶段八：WET 过滤脚本

1. 任务是什么

实现 `scripts/filter_wet_data.py`，让数据 pipeline 能从 WET 文件批量读取 record，执行语言过滤、质量过滤、有害内容过滤、PII masking，并输出过滤后的 JSONL 和统计信息。

2. 用到了什么大模型知识

真实预训练数据处理必须可批量化、可统计、可复现。过滤统计能帮助判断每一步是否过严或过松，也能分析最终训练数据的来源和质量。

3. 实验结果

脚本通过语法编译，可在有课程共享数据或本地 WET 文件时运行。完整 2,500 WET 文件实验未执行，原因是没有 `/shared-data/english-wet-data`，且本地磁盘空间不足。

10 阶段九：Tokenization 脚本

1. 任务是什么

实现 `scripts/tokenize_filtered_data.py`，使用 GPT-2 tokenizer 将过滤后的文本转换为 `np.uint16` 二进制 token 文件。

2. 用到了什么大模型知识

语言模型训练输入不是原始字符串，而是 tokenizer 输出的 token ids。文档间需要追加 EOS token，避免模型把两个独立网页错误地看成连续上下文。

3. 实验结果

脚本通过 `compileall` 语法检查。由于没有完整 filtered WET 数据，未生成完整训练 bin；但小规模样例和接口逻辑已经准备好。

11 阶段十：提交与最终验证

1. 任务是什么

整理报告、失败日志和提交包，确认公开测试全部通过，并调整打包规则，避免遗漏必要 fixture 或打入本地大文件。

2. 用到了什么大模型知识

大模型数据实验需要可复现记录：数据来源、过滤规则、失败原因、测试结果、资源限制都必须清楚。否则即使代码可跑，实验结论也难以复核。

3. 实验结果

最终验证:

```
python -m pytest -q
21 passed in 0.23s

python -m compileall cs336_data scripts tests/adapters.py
success

bash test_and_make_submission.sh
21 passed in 0.20s
```

提交包 `cs336-assignment4-submission.zip` 已生成。打包脚本也已调整，保留公开测试需要的 `.txt` fixtures，并排除本地 `local-shared-data/*`。

12 结论

本次 Data assignment 已经完成本地可实现和可测试的全部阶段。核心结论是：大语言模型的数据质量控制不是附属步骤，而是训练 pipeline 的关键部分。HTML 抽取、语言识别、PII masking、安全过滤、质量过滤、去重和 tokenization 会共同决定模型的训练效率、泛化能力、安全性和隐私风险。受限于没有课程共享数据、没有足够磁盘和没有 8 B200 GPU，本次没有运行完整 Common Crawl 过滤和训练实验；但对应代码、脚本、测试和提交包已经准备好。